DOI:10.19850/j.cnki.2096-4706.2025.22.003

# 基于 LangChain+Deepseek 的智慧燃气 GIS 领域问答系统研究与实现

石善忠
（上海城市地理信息系统发展有限公司，上海 200032）

**摘　要：**针对大语言模型在垂类应用中普遍存在的准确度不高的问题，文章采用 LangChain 作为核心开发框架，并集成 DeepSeek 先进大语言模型技术。对比并实践了 Ollama 与 vLLM 等前沿模型部署方案，最终采用 vLLM 实现生产环境下的高性能推理，解决了 Ollama 在并发场景下的性能瓶颈。通过模块化架构设计，构建了包含意图分析引擎、智能记忆系统、多智能体协同、输入输出处理链及自定义工具集成的完整技术体系。针对智慧燃气 GIS 系统的复杂业务场景，基于该技术体系打造了具备行业深度理解能力的高性能智能对话系统。

**关键词：**LangChain；DeepSeek；Ollama；对话系统
**中图分类号：**TP18；P209　　**文献标识码：**A　　**文章编号：**2096-4706（2025）22-0012-05

# Research and Implementation of an Intelligent Q&A System for the Smart Gas GIS Domain Based on LangChain+DeepSeek

SHI Shanzhong
(Shanghai City GIS Developing Co., Ltd., Shanghai 200032, China)

**Abstract:** To address the common issue of low accuracy in vertical applications of Large Language Models (LLMs), this paper adopts LangChain as the core development framework and integrates DeepSeek's advanced LLM technology. By comparing and implementing cutting-edge model deployment solutions such as Ollama and vLLM, the scheme ultimately adopts vLLM to achieve high-performance inference in production environments and effectively resolves Ollama's performance bottlenecks in concurrent scenarios. Through a modular architecture design, this paper develops a comprehensive technical framework that integrates an intent analysis engine, intelligent memory system, multi-agent collaboration, input/output processing chain, and customizable tool integration. To address the complex business scenarios of smart gas GIS systems, this framework is utilized to create a high-performance intelligent dialogue system with deep industry understanding.

**Keywords:** LangChain; DeepSeek; Ollama; dialogue system

## 0 引　言

目前，虽然人工智能与 GIS 的结合是学界和业界的热点，但将大语言模型技术深度融合于智慧燃气这类专业 GIS 平台，构建能够处理复杂业务逻辑的智能问答系统的研究与实践尚处于探索阶段。尤其在保证回答精准度、处理高并发请求等方面，仍面临诸多挑战。在此背景下，开发高效、高性能的智能 GIS 问答系统具有重要意义。目前，国内外基于 GIS 平台构建智能 AI 助手应用研究较少，对于构建智慧燃气 GIS 领域问答系统存在一些问题及挑战 [1-5]。针对上述问题本文利用 Langchain 作为核心开发框架，采用 Ollama 与 vLLM 的混合部署方案实现多模型的问答返回和并发处理，实现问答内容的精准度和响应速度。集成 RAG 知识库的检索提供相关操作手册的专业知识解答，同时构建 SQL Agent 智能体支持数据查询与可控数据的修改，利用 LCEL 链式（LangChain 表达式语言）实现从用户指令到大模型响应的全链路高效流转，并采用 Sqlite 异步存储方案实现用户会话和大模型输出的持久化跟踪。

## 1 系统设计

### 1.1 系统总体设计

系统架构主要分为意图分析引擎，专业知识库操作手册问答，燃气专业数据查询，可控业务数据修改，智能空间分析模块组成 [6-7]。整体架构如图 1 所示。

### 1.2 Langchain 和 Deepseek 介绍

LangChain 是一个专门为利用语言模型创建应用程序而设计的全面框架，LangChain 是一款面向开发者的前沿开源框架，专门为构建大语言模型 (LLM) 应用而设计。它集成了一系列实用的工具、

收稿日期：2025-05-19



中国知网 https://www.cnki.net

组件和接口，使开发者能够轻松构建高效、灵活的应用。

DeepSeek 是一款基于人工智能技术的大语言模型，旨在通过自然语言处理和深度学习算法，帮助用户更高效地获取和理解信息。它能够从海量数据中提取关键内容，提供精准的搜索结果，并支持智能问答、知识图谱构建等功能，广泛应用于教育、科研、商业等领域。

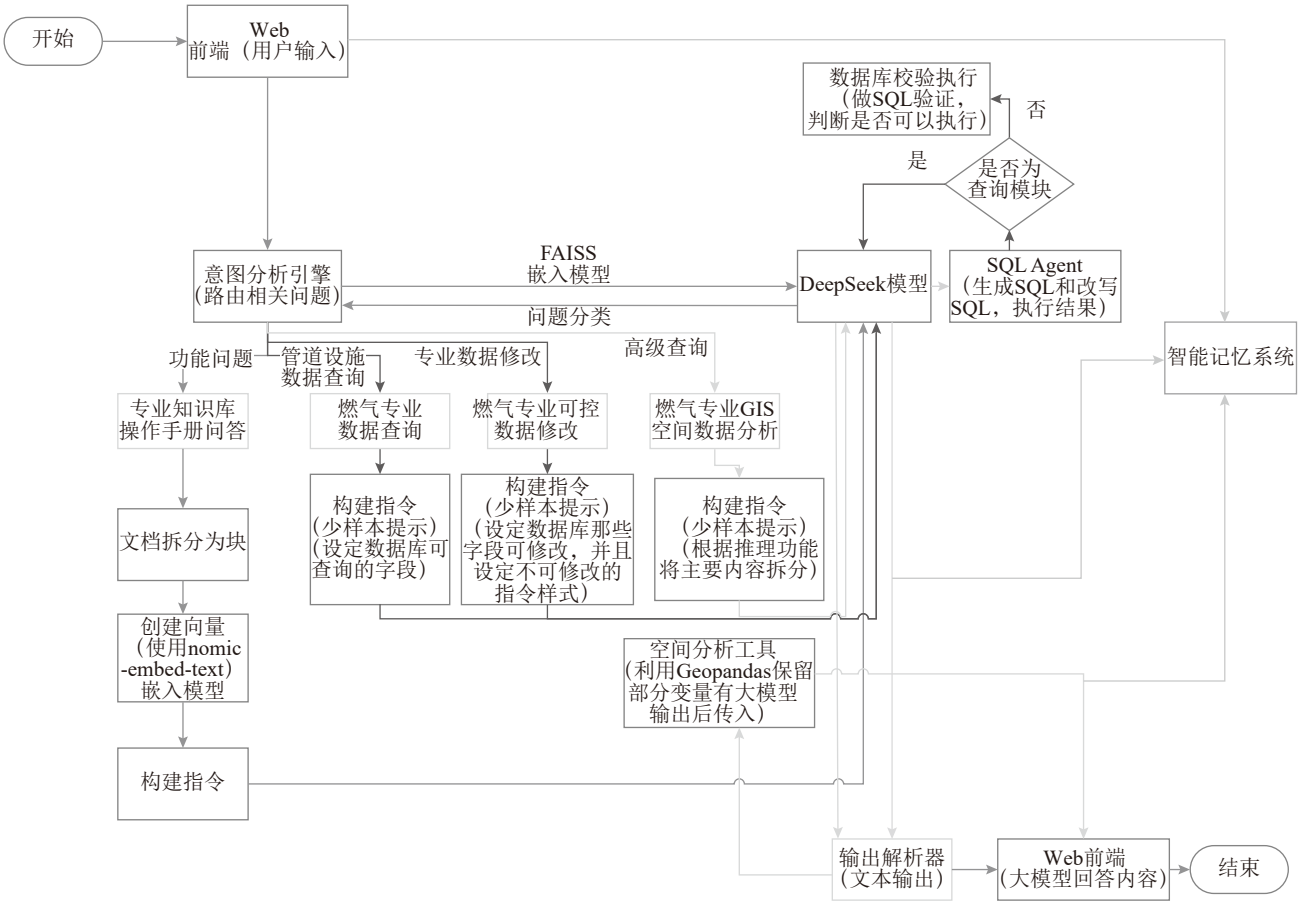


图 1　整体架构图

### 1.3　Ollama 和 vLLM 应用部署

Ollama 是一个专注于本地部署的大型语言模型服务框架，它旨在简化和优化大型语言模型在本地环境中的部署、管理和推理流程。

vLLM 是一个为大语言模型推理设计的高性能服务库。其核心优势在于采用了 PagedAttention 等先进技术，有效解决了内存碎片化问题，并实现了高吞吐量的连续批处理（Continuous Batch Processing），从而在处理高并发请求时表现远超传统推理引擎。

在基于 LangChain 的开发环境下，采用 Ollama 部署 DeepSeek 模型进行本地测试，发现其并发能力有限，在多人访问时会出现显著延迟，难以满足企业级应用需求。尽管 LangChain 提供的 ChatOllama 工具能便捷地加载 Ollama 托管的模型，但其仅适用于个人开发或小规模场景。因此，在正式生产环境中，对其进行了架构优化，将后端推理引擎切换至 vLLM，并重构部分代码以适配高性能推理需求。

Langchain 框架中可以采用 ChatOpenAI 的方法进行模型导入并且设定 api_key,model,base_url,tempeature 参数，在设定 base_url 中以例 :"http://ip:8000/v1" 进行 api 访问。这种代码构建方式可适配多种本地部署的大语言模型框架。下面为调用模型的核心代码：

```
from langchain_community.llms import Ollama
from langchain_oepnai import ChatOpenAI
from langchain_community.embeddings import OllamaEmbeddings
url="http://ip:8000/v1"
url2="http://ip:8000 "
model = ChatOpenAI(
api_key = "EMPTY",
  model="deepseek-r1:14b", # 模型名称
base_url=url, # 本地服务地址
  temperature=0
)
```



中国知网　https://www.cnki.net

emb =OllamaEmbeddings(model="nomic-embed-text",base_url=url2) # 加载嵌入模型

## 2　核心模块详情及关键技术

### 2.1　意图分析引擎

意图分析引擎作为智慧燃气 GIS 智能 AI 助手的核心组件，采用 Prompt 提示工程技术设计主要对提示词的开发和优化，用户输入内容可进行结构化的解析和意图识别，通过精心设计的提示词优化输入信息来引导人工智能生成高质量有针对性的回应。比如用户输入：“调压器这个设备有多少？”，意图分析引擎是通过代码中主要路由链条来识别问题方向进行分类，会将这个问题导向到设施查询功能模块。在整个助手模块中将模块分为设施查询，功能查询，管线查询，巡检数据查询，高级查询，空间查询，特定模块数据可控修改以及特定模块的查询。通过持续优化提示词，引导大模型高质量、有针对性地输出，确保意图分类的准确性。

#### 2.1.1　提示词工程

在智慧燃气 GIS 高性能智能对话系统中指令是非常重要的，Langchain 提供了多种技术方案本文主要采用零样本提示（Zero-shot Prompting）和少样本提示（Few-shot Prompting）。这两种方法显著提升了模型回答问题的准确度。利用 PromptTemplate 和 FewShotPromptTemplate 组件，构建了灵活的提示模板体系，能够有效指导模型完成各类专业任务。采用这种混合提示范式架构通过动态路由机制根据问题复杂度自动切换。

#### 2.1.2　语义相似度检索

采用了高级提示词技术，通过语义相似度匹配来优化对话系统的示例选择 [1]。具体实现中，使用 SemanticSimilarityExampleSelector 组件，结合 nomic-embed-text 嵌入模型和 FAISS 向量检索库，动态筛选与用户输入最相关的示例。设定参数时，需预先定义输入输出示例对，并配置 k=2 以返回最相似的 2 条结果，然后拼接在 FewShotPromptTemplate 组件中。这种基于语义相似度的少样本选择方法，能够精准匹配上下文，确保对话系统生成的回复既相关又富有洞察力，从而显著提升对话系统回答问题的质量。关键核心代码如图 2 所示。

通过少样本学习结合语义相似度检索技术，可显著提升大模型在意图分析过程中分类任务的准确率，通过添加更多的样例是数据模型的分类性能也有明显的提升，这种意图识别机制能精准的将用户问题分类到相应模块，确保系统的响应及时性。

```
example_prompt = PromptTemplate(input_variables=["input","output"],template="示例输入:{input}\n示例输出:{output}")#定义指令模版
examples=[{"input": "如果出现功能怎么用，网址是什么？，系统登录网址是什么？", "output":"功能问题"}, ] #指令样式设定入输入内容以及输出内容
example_selector = SemanticSimilarityExampleSelector.from_examples(
        examples,
        emb,
        FAISS,
        k=2, ) #examples属于每个示例包含输入和输出，ollamaembeddings属于向量模型，FAISS用于存储的向量检索库，k所要选择的最相似数量
prompt = FewShotPromptTemplate(
example_selector=example_selector, #用于选择示例的对象
example_prompt=example_prompt, #用于格式化示例的PromptTemplate实例
prefix="""以下是功能定义：
            1. 功能问题：这可能涉及系统或工具的操作方式，比如如何使用某个功能。
            2. 设施查询：对现有的设施数量、属性进行查询
            3. 管线查询：对现有的管线长度、属性进行查询
            4. 巡检数据查询：可能涉及检查记录等进行查询
            5. 高级查询：这似乎包括需要多参数组合的内容，比如巡视率或巡视情况，这些可能需要多个条件来生成结果。
            6. 特殊查询：涉及自定义范围的查询，如特定路名、小区名等。
            7. 老化管道项目数据查询：涉及到老化管道项目的查询比如ERP编号，实施年份。
            8. 老化管道项目数据修改：涉及到老化管道项目的工程状态修改。
            <question>
            {question}
            </question>"""
suffix="用户输入: {question} \n 输出: ", #作为后缀，输入变量的格式
input_variables=["question"],) #变量列表用于接受
```

图 2　利用少样本结合语义相似度检索的核心代码

#### 2.1.3　路由的构建

采用 LangChain 的 Runnable 链式组合机制，通过可执行对象的输入输出串联形成处理流水线。利用 RunnableLambda 封装自定义函数并嵌入流式处理，实现上一步输出到下一步的实时传递，使分类结果精准路由至对应功能模块执行。所以路由的创建在整个意图分析引擎模块中是至关重要的。相关关键代码如图 3 所示。

```
################路由转换#########################
#路由静态路由
def route(info):
    keywords_gx = ["管线查询"]
    keywords_ss = ["设施查询","巡检数据查询"]
    if "功能问题" in info["topic"]:
        return chainqa
    elif  any(keyword in info["topic"] for keyword in keywords_ss) :
        return final_chain_sql
    elif  any(keyword in info["topic"] for keyword in keywords_gx) :
        return final_gd_chain_sql
    elif "高级查询" in info["topic"]:
        return final_chain_split_xsl
    elif "特殊查询" in info["topic"]:
        return chain_zdy
    elif "老化管道项目数据查询" in info["topic"]:
        return final_chain_lhgd
    elif "老化管道项目数据修改" in info["topic"]:
        return final_chain_lhgdupdate
    return chainqa
#################################################
full_chain = {
            {"topic": onchaina, "question": lambda x: x["question"],"sessionid": lambda x: x["sessionid"]}
            | RunnableLambda(route)
            }
```

图 3　RunnableLambda 构建路由的关键代码

### 2.2　智能记忆系统

采用了基于 SQLite 的异步读写存储方案。考虑到系统需要记录用户会话信息，通过创建 sessionid 为核心的登录用户追踪机制。并设计了两个核心功能函数：一个用于存储用户问题，另一个来存储大模型生成的回答，以此构建了完整的对话记录查询体系，便于后续查询客户的问题以及回答内容的准确性，当然 Langchain 本身在 MessagePrompt 中有相关函数为 MessagesPlaceholder 方法来存储历史消息，有着记忆历史消息的功能，但在实际应用中发现，当模型首次回答出现错误时，该方法可能导致错误持续累积影响后续对话。因此决定放弃使用该内置功能，转为上述开发的基于 SQLite 的异步读写存储智能记忆系统，异步处理可以提高读写的效率。因此，本研究设计的基于 SQLite 的异步回话追踪系统，不仅实现了对话历史的持久化，更重要的是通过将用户提问与模型回答进行独立存储和管理，构建一种“无状态 - 有历史”（Stateless with History）的交互模式。该模式有效隔离了单次交互的错误，避免了错误信息的累积传播，显著提升了多轮对话的鲁棒性。


中国知网 https://www.cnki.net

## 2.3　SQL Agent 智能体

### 2.3.1　SQL Agent 智能体构建

学习少样本结合语义相似度检索的方法也可用于 SQL Agent 智能体构建中，将查询语句预定义在样例 examples 中，在提示词工程中采用混合模式进行构建，在 example_prompt 中需明确定义可查询的表，字段以及字段说明（如 company 为公司属性）。大模型指令工程是自然语言处理的关键环节。需要清晰界定可以处理以及不可以处理的内容，构建指令时可以用 ### 作为段落分隔符，明确标注不允许输入内容（如：查询表的投运日期字段）避免因为不可查询内容导致查询失败。指令必须使用简洁明确的语句，避免模棱两可的表达，防止 LLM 误解意图生成错误 SQL。

在构建 SQL Agent 智能体时，连接数据库需要使用 SQLAIchemy 驱动的 SQLDatabase 类与数据库建立接口连接，Langchain 中提供了 create_sql_query_chain 内置链可以高效的生成查询语句 SQL。成功生成相应 SQL 语句之后再使用 QuerySQLBaseTool 将查询执行功能整合在链中，最后将问题和答案再次传递到 LLM 中进行处理即可得到自然语言处理后的结果。在指令样本中可使用 partial 方法动态注入程序变量，实现参数化查询构建。

### 2.3.2　SQL 输出解析器改写

LLM 在生成 SQL 过程中容易出现一些错误，主要错误是多了一些关系数据库不可执行的冗余标记，如：```SQL ```,<think>,SQL query:, SQL:。可通过 Langchain 原生的输出解析器进行结合正则表达式精准匹配有效的 SQL 语句并清洗输出，从而提升 SQL 语句执行成功率。该方法具有多模型的适配性可针对不同的 LLM 生成的异常 SQL 采用这个方法进行修改，以下为 SQL 输出解析器改写的关键代码如图 4 所示，AI 助手生成的查询结果如图 5 所示。

```
class SqlQueryParser(StrOutputParser):
    def parse(self,text):
        pattern = r"```sql\n(.*?)\n```"  # 匹配代码块内容
        match = re.search(pattern, text, re.DOTALL)
        if match:
            sql_code = match.group(1).replace("SQL query:","").replace("SQL:","")
            print(sql_code)
            return sql_code
        return text

class StrReOutParser(StrOutputParser):
    def parse(self,text):
        result = re.sub(r'<think>.*?</think>', '', text, flags=re.DOTALL)
        return result
```

图 4　输出解析器改写

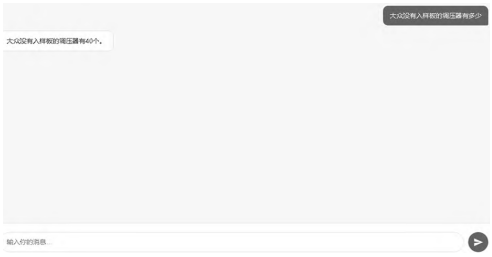


图 5　AI 助手生成的查询结果图

### 2.3.3　可控数据修改 SQL Agent 智能体构建

基于原有的 SQL Agent 的模式下，要对数据进行修改，由于构建 SQL 链时存在较高的风险。可能会生成错误 SQL（如生成没有带条件语句的 SQL）等，所以在处理可控数据修改时尤为谨慎。第一可提供有效示例和无效示例的对比。（如有效示例为：将预留编号 1352,1550, 1511 计划年份改为 2025。无效示例为：将预留编号 1352,1550,1511 改为 2025，将预留编号 1352,1550，计划年份改为 2025。当处理有效示例返回 SQL 语句，处理无效示例返回 None。）这样可限制大模型生成错误 SQL 将整体表格内容进行修改，第二强制检验机制，需要再执行 SQL 语句中添加不可执行 delete 以及 update 中没有 where 条件语句的。通过“指令级引导（Instruction-Level Guidance）”与“执行前检验（Pre-Execution Validation）相结合的双重安全保障机制”，本系统能有效防范因 LLM 生成错误 SQL（如无 Where 条件的 Update 或 Delete 语句）而导致的数据安全风险，确保了数据修改操作的精确性和安全性。可控数据修改是在整个 SQL Agent 模块中应用比较方便，可以将多数据，多字段信息通过智能体批量的将数据修改提高工作的效率。

## 2.4　智能空间分析模块

### 2.4.1　LCEL 链式传递

LangChain 通过其 LCEL（LangChain Expression Language）提供了高效的链式调用机制，利用直观的管道符语法即可快速构建 LLMChain，实现流畅的会话调用和上下文传递功能，无论是简单的“指令+LLM”链，还是复杂组合高度复杂的链，LCEL 都可以处理

将“指令 | 模型 | 输出”这种组合可以高效的生成相应的结果，也可以自定将工具融合在其中，通过使用调用 invoke() 的方法先将模型结论输出再利用 LCEL 的方式传递到自定义工具中，注意在处理自定义工具时候要把此工具变成 Runnable 类型才可以进行链式传递，可采用 RunnbaleLambda 函数进行包装即可。模型语义拆解关键代码如图 6 所示，使用自定义工具代码如图 7 所示，LCEL 链式传递代码如图 8 所示。

```
def split_tasks_zdy(input_text: str) -> dict:
    task_split_prompt = ChatPromptTemplate.from_template("""
    将以下问题请按照要求把关键字拆分出来：
    {question}

    返回格式：
    内容\n

    地址包含路名，小区名称等
    关键字：口径，材质，管道及设施，管道，设施，类型，道路，街坊，压力级制,管线
    道路，街坊本身就是独立的关键词，需要按照要求进行拆分
    #样式#
    如用户输入:查询西郊宾馆管道及设施，则输出是"西郊宾馆,管道及设施"。
    如用户输入:查询西郊宾馆管道及设施根据类型分组，则输出是"西郊宾馆,管道及设施,类型"。
    只要按照结构输出请按照字面意思转换，不要添加任何前缀和后缀，也不要修改关键词，按照上面示例进行拆分，不要添加任何关键字以外的内容
    双引号代表的结构样式，不需要输出,双引号里面内容需要输出
    请按照样式输出，不要进测试号\n
    内容两字不用输出，特殊意指四个字也不需要输出\n
    请按照示例要求拆解关键字，不需要添加或者修改关键词，不需要添加或者修改关键词，不需要添加或者修改关键词\n
    输出时候不要带空格,输出时候不要带空格，输出时候不要带空格\n
    """)

    split_chain = task_split_prompt | model | StrReOutParser()
    tasks = split_chain.invoke({"question": input_text})
    task_dict = {f"task_{i+1}": task for i, task in enumerate(tasks.split("\n"))}
    filtered_dict = {k: v.strip() for k, v in task_dict.items() if v.strip()}
    return filtered_dict
split_chain_zdy = RunnableLambda(split_tasks_zdy)
```

图 6　模型语义拆解



中国知网　https://www.cnki.net

```python
def run_parallel_tasks_zdy(tasks: dict) -> dict:
    global bp
    keywords_gx = ["管道及设施","管线及设施","管道","管线"]
    keywords_ss = ["调压器"]
    results = {}
    print("这里是run_parallel_tasks_zdy转接",tasks)
    for key, task in tasks.items():
        print(task)
        if any(keyword in task for keyword in keywords_gx):
            hs = task.split(",")
            print(hs)
            bp = selectall(hs)
            print(bp)
    return bp

parallel_chain_zdy = RunnableLambda(run_parallel_tasks_zdy)
```

图 7　用 RunnableLambda 方式封装自定义工具

```python
final_chain_split = (
    RunnableLambda(async_save_to_db_q)  # 传递原始输入
    | split_chain_zdy
    | parallel_chain_zdy
    |RunnableLambda(async_save_to_dbstr)
    | StrOutputParser()                # 解析为字符串
)
```

图 8　用 LCEL 方式进行链式传递

### 2.4.2　空间分析工具

GeoPandas 是一个开源项目，旨在向 Pandas 类添加对地理数据的支持 [8]。基于语义拆解的结果，利用 GeoPandas 构建的空间分析工具（如空间数据交叉计算等）可依托大模型实现智能化支持，这开创了自然语言处理与地理信息系统深度融合。利用 LCEL 链式传递的方式可将空间计算结果经输出解析器结构化处理后提供给前端会话页面。

## 3　测试结论分析

主要存在两方面的问题：

1）模型幻觉问题。大语言模型有时会生成看似合理但实际不准确的信息，这种被称为模型幻觉问题 [9]。在本文的知识库检索状态下，即使采用向量化方式匹配手册内容，当遇到文字近似度较高的情况时，仍可能出现幻觉问题。导致结果输出可能存在偏差，该问题主要是局限于语义相似度计算情况下对细微差异敏感性不够及向量匹配上下文的局限性。为缓解此问题，未来的优化方向可包括：引入重排（Re-ranking）模型对初步检索的文档进行二次排序，提升上下文的精准性。探索查询重写（Query Rewriting）技术，将用户的模糊问题分解或转化为更适合向量检索的子问题。对嵌入模型在燃气领域的专业语料上进行微调（Fine-tuning），增强其对行业术语的语义辨识能力。

2）模型精准度问题。大语言模型在处理专业计算过程（如巡检率和入样板率）或在比较模糊的语言（如巡检情况，设施总览）这种词时候会出现随意匹配到其他内容上去。针对专业计算（如巡检率）的精准度问题，未来的改进核心是工具调用（Tool Calling）。与其依赖 LLM 直接进行数学计算，更可靠方案是：训练的 LLM 识别计算意图，并生成调用预定好的函数。由确定性的代码来完成计算，LLM 仅负责理解问题和呈现结果，从而彻底解决计算不准的问题 [10]。

## 4　结　论

随着 AI 技术的持续发展，尤其是多模态大模型和自适应学习能力的提升，AI 结合 GIS 将在未来的城市规划，环境，灾害等领域发挥更大的作用，本文探讨了如何利用大语言模型增强 GIS 的工作流，包括自然语言的交互，空间计算自动化以及数据统计分析。通过 LangChain 的 LCEL 链式传递机制，实现了高效的传递和结构化的输出，使得复杂的地理信息分析能够更直观、更高效的呈现给用户。

**参考文献：**

[1] 张昭云 .LangChain 框架下基于 ChatGLM3 模型的个性化儿科健康咨询服务研究 [D]. 兰州：兰州财经大学，2025.

[2] 裴春营，廖芳芳，庞晓峰，等 . 基于高精度立体测绘卫星三维大模型建模技术研究与应用 [J]. 现代信息科技，2025，9（13）：144-148.

[3] 罗晶，肖德广，周阳 . 基于数字孪生的路桥一体化数字底座建设研究 [J]. 现代信息科技，2025，9（17）：143-146，151.

[4] 贾琳瑛 . 基于知识图谱的客站应急事件处置问答系统的设计与实现 [D]. 北京：北京交通大学，2024.

[5] 黄梦迪，陈敏 . 基于 LLM 和 RAG 的人才政策智能问答系统研究 [J]. 数字经济，2025（3）：38-41.

[6] 范路桥，段班祥，高洁，等 . 基于 Python+Flask+MySQL 的知宝问答系统 [J]. 现代计算机，2022，28（22）：93-98.

[7] 刘安骞 . 基于 LangChain 与 DeepSeek 的 JavaScript 教学辅助智能体的研究 [J]. 现代信息科技，2025，9（7）：76-82.

[8] 石善忠 .GIS 在城镇燃气管网中的关键技术及应用 [J]. 科技资讯，2024，22（6）：36-40.

[9] 陶晓英 . 基于混合架构的大语言模型智能问答系统研究 [J]. 邮电设计技术，2024（5）：48-55.

[10] 姚宏利，陈晓楠 . 基于 LLM 和 LangChain 的电力问答系统设计与实现 [J]. 无线互联科技，2025，22（13）：49-52+66.

**作者简介：**石善忠（1993—），男，汉族，上海人，工程师，本科，研究方向：地理信息系统、人工智能、大数据处理。



中国知网 https://www.cnki.net